

1교시: Docker 운영 관점 정리

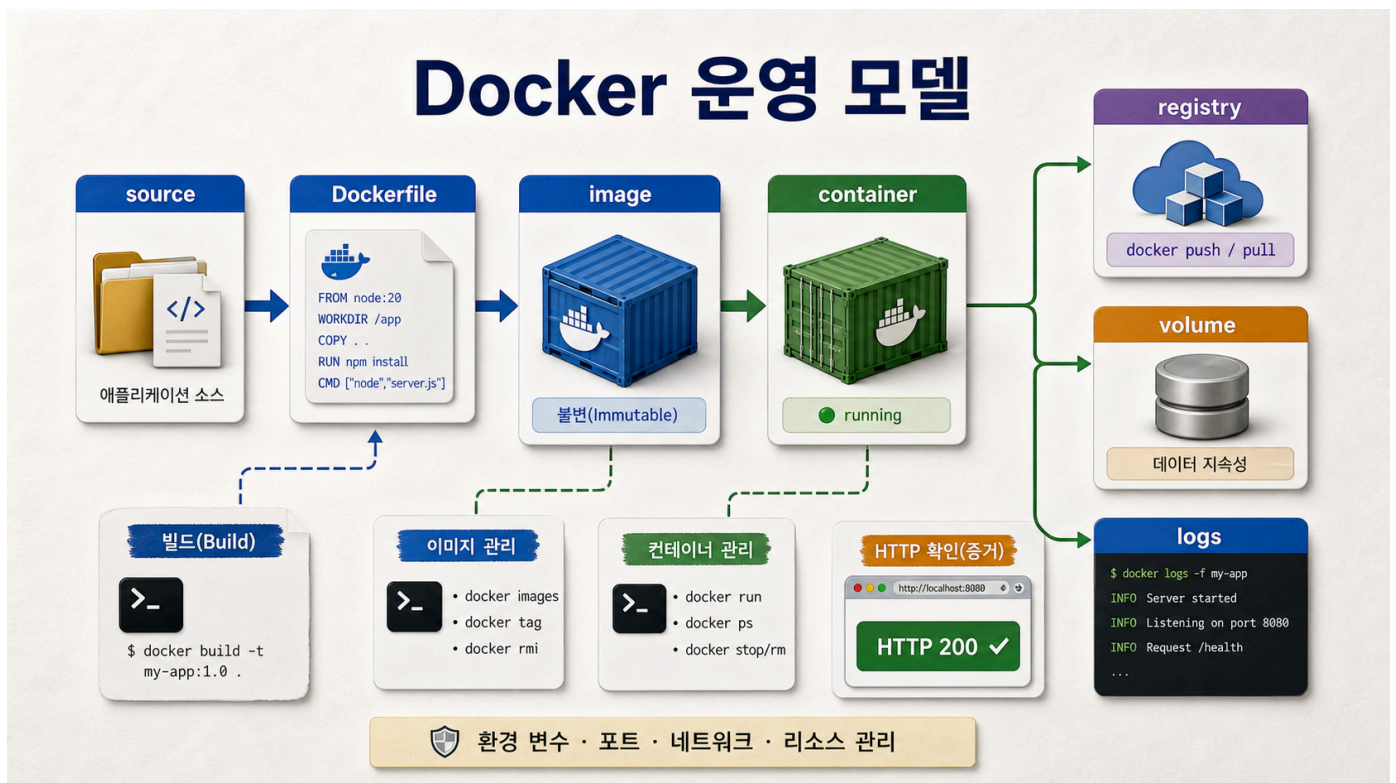
수업 목표

- container가 VM이 아니라는 점을 운영 관점에서 설명한다.
- stateless app, immutable image, persistent data의 경계를 구분한다.
- Week 2 산출물을 운영 evidence 기준으로 다시 읽는다.

50분 흐름

시간	활동	비중	산출
0-8분	Week 2 전체 복습	설명 15%	Docker concept map
8-20분	container vs VM 운영 차이	설명 25%	boundary note
20-32분	stateless/immutable/persistent 구분	설명 25%	state map
32-42분	Week 2 evidence inventory	실행 20%	evidence gap list
42-50분	Day 5 완료 기준 정리	설명 15%	completion note

Visual 1: Docker 운영 모델



이 visual은 image, container, registry, volume, log evidence가 운영 관점에서 어떻게 연결되는지 보여준다. 볼 지점은 container를 VM처럼 다루지 않고 image와 runtime state를 분리하는 것이다.

핵심 설명

container는 작은 VM이 아니다. VM은 guest OS를 포함하는 가상 머신이고, container는 host kernel을 공유하면서 process와 filesystem, network namespace를 격리해 실행한다. 이 차이는 운영 판단을 바꾼다.

VM처럼 container 안에 들어가서 수동 수정하고 오래 유지하는 방식은 Docker의 장점을 약하게 만든다. Docker에서는 image를 build artifact로 만들고, container는 그 image를 실행한 instance로 본다. 수정이 필요하면 container 안에서 고치는 것이 아니라 source와

Dockerfile을 바꾸고 image를 다시 build하는 방향을 기본으로 한다.

Day 5의 핵심은 이 사고방식이다. "내 container가 켜져 있다"가 아니라 "내 image는 어떤 기준으로 만들어졌고, 어떤 tag로 식별되며, 어떤 명령으로 실행되고, 어떤 evidence로 정상임을 증명하는가"를 말해야 한다.

선행 지식과 범위 경계

학생은 Day 1~4에서 Docker Desktop, image/container, Dockerfile, port, env, volume, network, Compose를 배웠다. 오늘은 그 조각을 운영 판단으로 통합한다.

오늘은 Kubernetes, ECS, ECR, CI/CD pipeline을 실제로 만들지 않는다. 대신 그 기술로 넘어가기 전에 Docker artifact가 갖춰야 할 최소 품질을 정리한다.

container와 VM 비교

항목	Container	VM
실행 단위	process 중심	OS instance 중심
filesystem	image layer + writable layer	virtual disk
시작 시간	일반적으로 빠름	상대적으로 느림
수정 방식	image rebuild 권장	instance 내부 변경 가능
운영 핵심	immutable image, reproducible run	machine lifecycle
오해 위험	VM처럼 오래 유지	image provenance 약화

stateless app과 persistent data

Docker 운영에서 가장 중요한 경계는 app process와 data lifecycle이다.

대상	권장 위치	이유
정적 app file	image layer	재현 가능한 artifact
runtime config	env/secret system	환경별 값 분리
log	stdout/stderr	수집과 관찰 용이
DB data	volume 또는 managed DB	container 삭제와 분리
secret	image 밖	노출과 재사용 위험 감소

Week 2의 static web app은 stateless에 가깝다. PostgreSQL data는 persistent data다. 이 둘을 같은 lifecycle로 다루면 cleanup 사고가 생긴다.

immutable image

immutable image는 한 번 만든 image를 실행 중에 수정하지 않는다는 운영 원칙이다. 완전히 절대적인 규칙은 아니지만, DevOps handoff에서는 강한 기본값이다.

좋은 흐름:

```
source change -> Dockerfile build -> tag -> run -> verify -> handoff
```

나쁜 흐름:

```
container exec -> inside file edit -> works on my machine -> no reproducible artifact
```

Week 2 evidence inventory

학생은 자기 repository에서 다음을 찾는다.

Evidence	Path/Command	Status
Dockerfile		complete/partial/missing
image tag	docker images	complete/partial/missing
docker run evidence	docker ps, curl	complete/partial/missing
Compose evidence	docker compose ps	complete/partial/missing
README handoff	README section	complete/partial/missing
failure RCA	note/README	complete/partial/missing
cleanup note	README	complete/partial/missing

학술 기준 연결

ABET 관점에서 이 교시는 문제 분석과 전문적 책임을 다룬다. container를 VM처럼 다룰 때 생기는 재현성 문제를 분석하고, data/secret/cleanup 책임을 설명한다.

CS2023 관점:

범주	적용
Knowledge	container, image, registry, volume, immutable artifact
Skill	evidence inventory와 gap list 작성
Disposition	작동 여부보다 재현성과 책임을 우선

실무 insight

현업에서 Docker 장애는 "Docker가 어렵다"보다 경계가 불명확해서 생기는 경우가 많다. image에 secret을 넣었는지, container 안에서 수동 수정했는지, DB data를 volume에 뒀는지, cleanup 명령이 data 삭제를 포함하는지 같은 경계가 중요하다.

운영자는 container 실행만 보는 것이 아니라 artifact provenance, runtime config, observability, rollback 가능성까지 본다.

오해 점검

오해	교정
container는 작은 VM이다	process와 filesystem 격리를 사용하는 실행 단위다
실행 중인 container를 고치면 된다	재현 가능한 image rebuild가 기본이다
stateless app에는 volume이 항상 필요하다	필요한 data lifecycle이 있을 때만 쓴다
logs는 container 안 파일에만 남긴다	stdout/stderr가 기본 관찰 경로다
Dockerfile이 있으면 운영 준비 완료다	run/check/cleanup/RCA가 필요하다

기록 템플릿

Lesson 1 Docker Ops Model

- Container vs VM:
- Stateless part:
- Persistent data:
- Immutable image evidence:
- Runtime config:
- Logs/evidence:
- Remaining gap:

평가 기준

기준	2점 evidence
운영 모델	container/image/runtime/data 경계를 설명했다
evidence	Week 2 산출물 gap list를 작성했다
책임	secret/data/cleanup 위험을 언급했다

전이 과제

Week 3 MSA에서는 service가 여러 개로 늘어난다. 학생은 다음 질문에 답한다.

서비스가 1개일 때도 image, config, port, log, data 경계를 명확히 못 하면, 서비스가 4개로 늘어났을 때 어떤 문제가 커질까?

공식 근거 링크

- Docker overview: <https://docs.docker.com/engine/docker-overview/>
- Twelve-Factor App: <https://12factor.net/>

운영 판단 심화

Docker 운영 관점에서 가장 먼저 나누어야 할 것은 artifact와 instance다. image는 artifact이고 container는 instance다. artifact는 review, tag, scan, push, pull 대상이다. instance는 start, stop, logs, health, cleanup 대상이다.

흔동	결과
container 내부 수정	source와 image에 반영되지 않아 재현 불가
tag 없는 image 사용	어떤 artifact를 실행했는지 설명 불가
log 위치 미기록	장애 시 관찰 지점 상실
volume 의미 미기록	cleanup 중 data 삭제 위험
config를 image에 고정	환경 변경마다 rebuild 필요

Lesson 1 Exit Ticket

Exit Ticket

- Docker를 VM처럼 다루면 안 되는 이유:
- image와 container를 구분하는 내 문장:
- 내 산출물에서 persistent data가 있는지:
- 다음 교시 Dockerfile review에서 확인할 것:

운영 판단 심화

Docker 운영 관점에서 가장 먼저 나누어야 할 것은 artifact와 instance다. image는 artifact이고 container는 instance다. artifact는 review, tag, scan, push, pull 대상이다. instance는 start, stop, logs, health, cleanup 대상이다.

이 구분이 흐려지면 다음 문제가 생긴다.

혼동	결과
container 내부 수정	source와 image에 반영되지 않아 재현 불가
tag 없는 image 사용	어떤 artifact를 실행했는지 설명 불가
log 위치 미기록	장애 시 관찰 지점 상실
volume 의미 미기록	cleanup 중 data 삭제 위험
config를 image에 고정	환경 변경마다 rebuild 필요

실무 runbook 관점

운영 문서는 명령어 목록이 아니라 의사결정 문서다. 아래 질문에 답해야 한다.

- 1. 지금 실행 중인 것은 어떤 image tag인가?
- 2. 정상 상태는 어떤 HTTP status 또는 log로 확인하는가?
- 3. container를 지우면 data도 지워지는가?
- 4. 같은 image를 다른 port로 실행할 수 있는가?
- 5. 장애가 나면 build 문제와 runtime 문제를 어떻게 나누는가?

Week 2 산출물 재해석

산출물	운영 해석
Dockerfile	build artifact 생성 계약
.dockerignore	build context와 secret risk 통제
image tag	artifact 식별자
docker run	단일 instance runtime 계약
Compose	multi-service local runtime 계약
README	handoff와 operational readiness
RCA	incident learning evidence

강한 설명 예시

이 앱의 정적 파일은 image layer에 포함되어 stateless하게 배포된다. runtime 설정은 host port publish로 주입되며, 현재 실습에는 persistent data가 없다. 정상 상태는 HTTP 200과 body marker로 확인한다.

추가 활동: 경계 표시

경계	내 앱에서의 예
build artifact	
runtime instance	

runtime config	
persistent data	
observability evidence	
cleanup target	

종료 전 확인

image는 ____이고, container는 ____이다.
container 내부에서 수정한 내용은 ____에 자동 반영되지 않는다.
운영 handoff에는 run 명령뿐 아니라 ____와 ____가 필요하다.